

FLOW APP - ONE PAGE SUMMARY

WHAT IT IS

Flow (@summoniq/flow, "SummonIQ Flow") is a Next.js + Electron app for agency operations. It unifies CRM, projects, scheduling, billing, and AI tooling.

WHO IT'S FOR

Primary persona: an agency owner/operator managing clients, delivery, and revenue.

WHAT IT DOES

- Shows a command-center dashboard for inbox, meetings, projects, billing, clients.
- Stores CRM entities: clients, contacts, projects, and related activity.
- Tracks execution with tasks, milestones, tickets, teams, and routing metrics.
- Supports billing workflows: invoices, payments, outstanding, and overdue totals.
- Surfaces calendar/email status through app APIs and status bar indicators.
- Provides desktop actions via Electron (open editor/folder/terminal, launch apps).
- Runs MCP tools through an in-app MCP client/server integration.

HOW IT WORKS (REPO EVIDENCE)

- UI: Next.js App Router pages/components inside AppShell navigation.
- Desktop wrapper: Electron main process + preload IPC bridge.
- App logic: server actions and API routes aggregate and mutate domain data.
- Orchestration: ProjectOrchestrator + TechLeadRouter + AgentPool + ContextManager behind /api/orchestration routes for analysis, routing, and assignment.
- MCP: /api/mcp/status uses MCPClient, which spawns lib/mcp/server.ts.
- Data: Prisma Client (PostgreSQL adapter) over prisma/schema.prisma models.
- Production hosting/infrastructure details: Not found in repo.

Data flow: User -> Next.js UI -> API/server actions -> Prisma -> PostgreSQL

Data flow: Electron UI -> preload IPC -> electron/main.cjs -> local fs/process ops

HOW TO RUN (MINIMAL)

1. bun install
2. Configure .env (DATABASE_URL required; NEXT_PORT defaults to 30140)
3. bun run db:generate && bun run db:push
4. bun run dev
5. Open <http://localhost:30140> (Electron dev window starts from the dev script)